LG 마블

기술 심층 분석 · 임베디드 Qt 멀티플레이 보드게임

EMBEDDED · QT · LINUX · MULTIPLAYER

C반 1팀 · **LG 마블** · 황선영 · 이도훈 · 허승회 · 조규현

CONTENTS · 6 SECTIONS

오늘 발표는 이렇게 진행됩니다

주제 정의부터 팀 회고까지, 두 가지 축으로 구성했습니다 — 기술 증명과 회고.

FOCUS 발표의 중심은 **03 · 핵심 기술** — 5장 분량으로 지텐 분석

INDEX 15 SLIDES

INDEX 06 CHAPTERS · 14 SLIDES

01 **주제 정의 · 결과 요약** — P. 03
한 판의 부루마블, 4대의 임베디드 보드

02 **개발 목표 · 결과** — P. 04-05
4가지 도전 과제와 달성 지표

03 **핵심 기술 · 5장** — P. 06-11
임베디드 · 드라이버 · 네트워크 · 맵 · 최적화

04 **결과 분석 · 기대 효과** — P. 12
정량 지표와 재사용 가능성

05 **향후 연구 과제** — P. 13
V4L2 · GPU · 무선 · 다인 확장

06 **수행 후기 · Q&A** — P. 14-15
배운 점, 어려웠던 점, 팀워크

CHAPTER 01

부루마블, LG 답게 다시 만들다

임베디드 보드 위에서 네 대가 동시에 돌아가는 실시간 멀티플레이 보드게임.
게임은 결과이자, **풀스택 임베디드 엔지니어링의 증거**입니다.

" FROM KERNEL TO UI "

네 가지 도전 과제

프로젝트 시작 시점에 정의한 4대 목표.
기술적 난이도와 프로덕트 완결성을 함께 고려했습니다.

01 임베디드 시스템 통합

DTB · RFS · Buildroot

전원만 켜면 게임이 실행되는 완결형 시스템. OS 부팅 수준까지 직접 손대야 하는 영역입니다.

02 디바이스 드라이버 활용

I2C · GPIO · sysfs

가속도 센서 · GPIO · LED를 게임 메커닉으로 활용합니다. 터치 이상의 물리 인터랙션을 확보합니다.

03 네트워크 시스템 구현

UDP · TCP · Host-Auth

4 대가 같은 게임 상태를 공유합니다. 방 탐색 자동화와 공정성 보장이 핵심 과제입니다.

04 맵 디자인 · 렌더링 최적화

QPainter · Isometric

GPU 없이 아이소메트릭 3D 보드판을 그리고 30 FPS 를 유지하는 렌더링 파이프라인.

두 트랙으로 쌓아올린 LG 마블

인프라·HW 통합은 앱과 병렬로,
앱은 네트워크 → 로직 → UX → 성능·3D 순으로 실기 검증하며 쌓았습니다.

DEV TRACKS

2 PARALLEL · v0 → v5

FOUNDATION TRACK

앱 버전과 **병렬**로 진행

초기 인프라

ARM 크로스컴파일 + RFS 자동 실행 후크 · `.dts` I2C 노드 · Buildroot 툴체인

HW 통합

GPIO 버튼 · I2C 가속도 센서 · LED `/dev/mem` mmap · ALSA 오디오

APP VERSION TIMELINE

v0 → v5 · 4 단계

v0.x — **멀티플레이 네트워크 환경 구축**

UDP 방 탐색 + TCP 호스트–클라이언트 · Lobby 동기화 · 본인 턴 주사위 검증

v1.x — **게임 로직 구현**

매각/파산 + 총자산 · 무인도 · 황금열쇠 · 미니게임 보상 · 통행료 면제권

v2 · v3 — **비주얼 UI & 감성 UX**

찬스카드 그래픽 · 이모지 리액션(상·하단 자동 배치) · `MSG_EMOJI` 4인 실시간 전파

v4 · v5 — **터치 최적화 & 성능·3D 렌더링**

CPU governor `600→1.2GHz` · 가상 키보드(한·영·숫자) · 애니메이션 14종 + 이징 4종 · QPainter 등각투영 3D · dirty-rect + 오프스크린 백버퍼

03 · CORE TECHNOLOGIES

앞으로 소개할 5 가지 핵심 기술

커널부터 UI까지 — 각 기술은 "필요(NEED) → 선택(CHOICE) → 구현(HOW)" 순서로 이어집니다.

01 임베디드 시스템 통합

Device Tree · RFS Hook · Buildroot

전원만 켜면 게임이 돌아가는 **완결형 보드**를 만들기 위해 OS 초기화부터 직접 다루었습니다.

02 디바이스 드라이버 활용

I2C 센서 · GPIO · sysfs LED

터치 이상의 물리 인터랙션 — **가속도·GPIO·LED**를 게임 메커닉으로 끌어왔습니다.

03 네트워크 시스템

UDP 탐색 · TCP Host-Auth · Lockstep

상대 IP 없이 방을 찾고, **4대가 한 프레임도 어긋나지 않는** 게임 상태 동기화.

04 어플리케이션 맵 디자인

QPainter · Isometric · Tile Bake

GPU 없는 RPi3에서 **QPainter 등각투영**으로 3D 보드판의 입체감을 구현했습니다.

05 성능 최적화

CPU Governor · ×2.0 Perf

앱 레벨 최적화의 한계를 **커널 governor 3줄**로 돌파했습니다 — 600MHz → 1.2GHz.

READING GUIDE

NEED · CHOICE · HOW

게임을 돌리기 전에, 보드부터 준비했습니다

전원을 켜면 자동으로 게임이 실행되고, OS가 가속도센서를 인식하도록 — **앱만 배포해서는 해결 불가능한 "시스템 영역"** 문제였기에 OS 초기화 단계부터 직접 손보았습니다.

1 필요했던 것

NEED

전원을 켜면 자동으로 게임이 실행될 것

로그인도, 터미널도, 데스크톱도 없이 — 전원 = 게임

운영체제가 가속도센서를 인식할 것

커널이 모르는 하드웨어 = 앱에서도 접근 불가

2 우리의 선택

CHOICE · WHY

Device Tree + RFS 커스터마이징 + Buildroot

WHY

앱만 배포해서는 해결 불가능한 문제. **OS 초기화 단계부터 손봐야 하는 "시스템 영역"** 문제였기 때문에, 커널이 읽는 **하드웨어 선언(DTB)**과 **부팅 스크립트(RFS)**를 직접 고치는 것이 유일한 길이었습니다.

3 어떻게 구현했나

HOW · 2 STEPS

01 DTB (.dts) — I2C 노드에 센서 추가

→ 커널이 부팅 시 센서를 자동 인식

02 RFS (S50local) — 부팅 훅에 앱 실행 명령 주입

→ 전원 ON = 게임 실행 (shell 없이 키오스크 부팅)

계층 스택 · BOOT ORDER

HW	→ Raspberry Pi 3 + MPU-6050
Kernel + DTB	→ 센서 노드 인식
input / IIO / ALSA	→ 표준 인터페이스 노출
RFS / S50local	→ Qt 앱 자동 실행
Qt App	→ LG 마블 게임 시작

터치로는 부족해서, 물리 입력을 끌어왔습니다

주사위 던지는 "손맛", 리듬 미니게임의 정확한 타이밍, 파산·승리의 즉각 피드백 — 모두 **리눅스 표준 인터페이스(I2C · GPIO · mmap · ALSA)** 위에서 해결했습니다.

1 필요했던 것

NEED

🎲 주사위 던지는 **"손맛"**

터치 버튼으로는 재현 불가능한 물리 감각

🎯 리듬 미니게임의 **정확한 타이밍 입력**

ms 단위 반응성 · 난이도별 차등 보상

💡 파산·승리의 **즉각 피드백**

화면 밖에서도 느껴지는 물리 신호

🔊 **배경음악 · 효과음**으로 몰입감 강화

주사위·결제·승리 순간의 청각 피드백

2 우리의 선택

CHOICE · WHY

I2C 가속도센서 + GPIO 버튼

LED (mmap) + ALSA 오디오

WHY

모두 **리눅스 표준 인터페이스** — 커스텀 드라이버 없이 Qt 앱에서 즉시 접근 가능. 커널 업데이트에도 호환성이 유지되고, sysfs·ioctl 패턴만 알면 **다른 프로젝트에 그대로 이식**할 수 있습니다.

3 어떻게 구현했나

HOW · 4 CHANNELS

🎲 **I2C 가속도센서** — 주사위 던지기

→ 2축 peak-to-peak · sign-change 감지

🎯 **GPIO SW2/SW3** — sysfs 50ms 폴링

→ 난이도별 보상: Easy 1 · Normal 2 · Hard 3

💡 **LED /dev/mem mmap** — GPSET0 / GPCLR0

→ 승리 GPIO6 점등 · 파산 GPIO5 점등

🔊 **ALSA aplay** — 서브프로세스 재생

→ --buffer-size=16384 로 언더런 방지

4대가 같은 게임 상태를 공유하도록 연결했습니다

상대 IP 없이 방을 찾고, TCP로 명령을 누락 없이 전달하고, 모든 단말이 동일한 시뮬레이션 결과를 진행하도록 — **UDP 탐색 + TCP Host-Auth + Lockstep**의 3단 구조로 해결했습니다.

1 필요했던 것

NEED

같은 LAN의 다른 보드를 자동 탐색

사용자가 IP를 알 필요 없는 방 찾기

4대가 완전히 동일한 게임 상태

주사위·위치·자산까지 한 바이트도 어긋나지 않게

클라이언트 조작에도 공정성 보장

치팅 방지 + 판정의 단일 진실원

2 우리의 선택

CHOICE · WHY

UDP 탐색 + TCP Host-Auth

Lockstep

WHY

UDP — 상대 IP를 모를 때 브로드캐스트만이 유일한 방법. **TCP + Host 판정** — 상태는 한 바이트도 놓칠 수 없고 치팅도 봉쇄. **Lockstep** — 주사위 seed 공유로 애니메이션 프레임까지 동일해집니다.

3 어떻게 구현했나

HOW · 4 STEPS

01 UDP :12346 — 방 탐색

→ 서브넷 브로드캐스트로 방 존재를 주기적으로 선포

02 TCP :12345 — 세션 연결

→ JSON 프로토콜로 게임 상태 송수신 (length-prefix)

03 Host 판정 — 단일 진실원

→ 모든 규칙 판정을 호스트가 수행하고 결과만 broadcast

04 Lockstep — diceSeed 공유

→ 동일 seed로 4대가 프레임 단위 동일 애니메이션 재생

실제로 주고받은 메시지는 이렇게 생겼습니다

4대가 같은 게임 상태를 공유하려면 어떤 메시지를 언제 누가 보낼지 **스펙으로 고정**되어야 합니다.
탐색(UDP) · 로비 · 게임 · UX · 미니게임 **5개 카테고리, 총 16종**으로 정리했습니다.

16 MESSAGES

5 CATEGORIES

UDP 탐색
2 MESSAGES

- `MSG_DISCOVER` — 클라이언트 → 호스트 방 탐색
- `MSG_ROOM_INFO` — 호스트 응답 · 방 정보 브로드캐스트

LOBBY 로비
5 MESSAGES

- `MSG_JOIN_ROOM` — 클라이언트 닉네임 등록
- `MSG_JOIN_ACCEPT` — 호스트 승인 + PlayerId 부여
- `MSG_PLAYER_LIST` — 로비 플레이어 목록
- `MSG_READY` — 준비 상태
- `MSG_PLAY_START` — 게임 시작

GAME 게임 진행
4 MESSAGES

- `MSG_TURN_START` — 턴 전환 (호스트 발행)
- `MSG_DICE_RESULT` — 주사위 결과 전파
- `MSG_ACTION_REQUEST` — 구매 · 건설 · 매각 · 우주여행 액션
- `MSG_GOLDEN_KEY` — 카드 드로우 결과 (호스트 echo)

UX 리액션
1 MESSAGE

- `MSG_EMOJI` — 실시간 이모지 리액션

MINIGAME 미니게임
4 MESSAGES

- `MSG_MINIGAME_START` — 미니게임 진입
- `MSG_MINIGAME_STATE` — 아이콘 위치 · 페이즈 실시간 중계
- `MSG_MINIGAME_RESULT` — 미니게임 종료 결과
- `MSG_MINIGAME_REWARD` — 보상 1~3장

GPU 없는 보드 위에, 3D 맵을 그렸습니다

RPi3 linuxfb 환경에서는 GPU 가속이 불가. **QPainter 등각 투영(isometric)**으로 원근 계산 없이 3D 보드판의 입체감을 CPU만으로 구현했습니다.

1 필요했던 것

NEED

부루마블 특유의 3D 보드 입체감

별장 → 빌딩 → 호텔의 시각적 성장

14종의 생동감 있는 애니메이션

주사위 · 이동 · 구매 · 파산 연출

4대에서 완전 동일한 화면

피어 간 결정적 렌더링

2 우리의 선택

CHOICE · WHY

QPainter 등각투영 + Static Bake

Lockstep Render

WHY

RPi3 linuxfb는 **GPU 가속이 불가**. QPainter는 Qt 기본 내장이라 안정적이며, **등각 투영은 원근 계산 없이 3D 착시**를 만들어 CPU 부담이 적고 타일 단위로 관리하기 쉽습니다.

3 어떻게 구현했나

HOW · 4 STEPS

01 타일 렌더링 — QPolygonF 4-vertex
→ u/v/height/flip 파라미터로 3D 건물 원근 구현

02 건물 레이어 — 3단계 스프라이트
→ 별장 → 빌딩 → 호텔을 z-order로 적재

03 Static Bake — 정적 요소 1회 렌더
→ QPixmap에 굽고 매 프레임 blit만 수행

04 Lockstep — diceSeed 공유
→ 4대 피어에서 프레임 단위 동일 재생

앱 최적화의 한계를, 커널 3줄로 돌파했습니다

Qt 레벨 최적화(dirty region · pixmap 캐시)로는 30 FPS 유지 불가.
원인은 하드웨어가 아니라 **보드 기본 governor가 powersave로 설정되어 CPU가 600MHz에 고정**되어 있었던 것 - 시스템 설정 문제였습니다.

1 필요했던 것

NEED

- 30 FPS · **프레임 예산 33ms** 안정 유지
  끊김 없는 주사위·이동 애니메이션
- 실기 **버퍼링 · 끊김 해소**
  VM 환경이 아닌 실제 RPi 하드웨어에서
- ALSA **오디오 언더런 방지**
  버퍼를 채우지 못해 튀는 현상 제거

2 우리의 선택

CHOICE · WHY

powersave → **performance**

CPU Governor (sysfs)

WHY

Qt 앱 레벨 최적화(dirty region · pixmap 캐시)로는 한계. 원인 분석 결과 **보드 기본 governor가 powersave → CPU가 600MHz에 고정**되어 있었고, 이는 하드웨어 한계가 아닌 시스템 설정 문제였습니다.

3 어떻게 구현했나

HOW · 2 STEPS

01 **governor 전환** — powersave → performance
→ 4개 코어 모두 sysfs에 한 번에 write

02 **S50local에 배치** — 부팅마다 자동 적용
→ 재부팅 후에도 설정 유지

RESULT · BEFORE → AFTER

~~600~~ MHz
powersave

→

1.2 GHz
performance · ×2.0

04 · REQUIREMENTS DELIVERED

요구사항을 빠짐없이 구현하다

기능 · 비기능 요구사항 전 항목 달성

기능 요구사항

FUNCTIONAL

게임 규칙

- 주사위 2개 · 더블 · 3연속 → 무인도
- 통행료 · 독점 · 3단계 건물 (별장 · 빌딩 · 호텔)
- 월급 (통과 30만 · 도착 60만) · 턴 타이머
- 건물 구입 · 매각 · 강제 인수
- 파산 로직 · 최후 생존자 승리

특수 타일 · 이벤트

- 무인도 로직 (3턴 대기)
- 황금열쇠 로직 (카드 덱)
- 재산세 (보유 자산 비례)
- 우주여행 로직 (순간이동)
- 리듬 미니게임 (3단계 난이도)
- 소득세 (현금 비례)

하드웨어 인터랙션

- 가속도 센서 주사위
- LED 물리 피드백 · 터치 UI
- GPIO 버튼 미니게임
- ALSA 오디오 (BGM + SFX)

네트워크 기능

- 4인 실시간 멀티플레이 · UDP 방 자동 탐색
- 이모지 리액션 · 호스트 판정

비기능 요구사항

NON-FUNCTIONAL

성능

- 30 FPS 안정 (프레임 예산 33 ms 준수)
- ALSA 언더런 0건
- 부팅 → 게임 실행 자동

공정성 · 안정성

- Host-Authoritative (규칙 판정 단일 권위 → 치팅 불가)
- Lockstep Deterministic (애니메이션까지 동일 재생)

운영 · 배포

- 부팅 자동 실행 (S50local)
- VM (x86_64) + 보드 (aarch64) 듀얼 빌드 환경

05 · FUTURE WORK

향후 연구 과제

단기 · 중기 2단계 로드맵

• PHASE 01 · SHORT-TERM

곧 적용 가능한 확장

1–3 MONTH

V4L2 카메라 통합

승리 사진 · 플레이어 얼굴 스티커

I18N 다국어 지원

한·영 리소스 분리 · QTranslator

SFX 효과음 · BGM 확장

게임 몰입감 강화

• PHASE 02 · MID-TERM

시스템 업그레이드

3–6 MONTH

EGL eglfs GPU 가속

소프트웨어 렌더 한계 해소 · 60FPS

NET 무선 네트워크 플레이

사내망 외부 플레이 · 세션 리커넥트

OTA 원격 OTA 업데이트

4대 동시 배포 · 롤백 구조

프로젝트 수행 후기

팀원 전원의 공통된 감상

01

배운 점

"같은 문제를 여러 계층에서 풀어볼 수 있다는 것"

- 커널 · 드라이버 · 앱의 경계에서 발생하는 문제 해결 경험
- sysfs 3 줄로 2× 성능을 내는 — 추상화의 힘
- 네트워크 · 하드웨어 · 렌더링의 공통 언어는 "측정"

02

어려웠던 점

"보드 4 대에서만 재현되는 버그"

- 단일 보드 · 시뮬레이터에서는 잡히지 않는 타이밍 이슈
- GPU 없는 환경에서 애니메이션 품질 타협점
- 로컬 개발 · VM 빌드 · 실기 실행의 3단 디버깅 파이프라인

Q & A

LG 마블 · 발표를 마칩니다 · Thank you

C반 1팀 · LG 마블 · 황선영 · 이도훈 · 허승회 · 조규현